Anomaly Detection in Azure ML Studio

This is an anomoly detection mechanism that contains multiple, small bits of code written in a programming language called, Python. It will interface with Microsoft's Azure AI Machine Learning cloud resources to obtain the credit card fraud dataset to conduct its analysis. The end result is to detect rare patterns in the data which in turn will allows humans to analyze and action on these anomolies.

The workflow integrates two environments:

1. **Azure AI Machine Learning Studio** – used for managing datasets, compute resources, and registered models.
2. **Python (Jupyter Notebook)** – used for data processing, model training, evaluation, and visualization.

We use the Credit Card Fraud Detection dataset (from Kaggle), which contains anonymized transaction features and a label identifying whether a transaction was fraudulent.

In brief, the Workflow will execute as follows:

Azure ML Components Used in This Workflow

Azure ML Component	Role in the Workflow
Azure ML Workspace	Central environment that stores datasets, experiments, compute resources, and registered models.
Azure Data asset (creditcard_fraud)	A managed dataset stored in the workspace that contains the credit card transaction data.
Jupyter Notebook (Authoring → Notebooks)	Interactive environment used to run Python code for data processing, model training, and analysis.
Azure ML SDK (azureml-core)	Python library used to connect the notebook to Azure ML services such as datasets and model registry.
Compute Environment	The runtime where the notebook executes code (local machine or Azure ML compute instance).
Model	Stores the trained model so it can be reused.

Workflow

This notebook will walk through the following machine learning stages.

Step 1: Connect to the Azure Cloud

Intent: Install Python libraries to prep the Jupyter environment. Connect/authenticate to the Azure environment so that Jupyter can interact with the fraud data set.

Expected Outcome: All required packages are available and the environment is ready to load data, train a model, and evaluate results.

In [ ]:
```
# Step 1: Import Packages and Connect to your Azure Workspace
#!pip install setuptools
!pip install azureml-core
from azureml.core import Workspace, Dataset          # see https://pypi.org/project/
import pandas as pd                                  # see https://pandas.pydata.org
from sklearn.ensemble import IsolationForest         # see https://scikit-learn.org/
from sklearn.metrics import classification_report    # see https://scikit-learn.org/
from azureml.core.model import Model                 # see https://docs.microsoft.co
```

Step 2: Load the Credit Card Fraud Dataset from Azure ML

Intent: Connect to the Azure Machine Learning workspace and retrieve the credit card fraud dataset.

Expected Outcome: The dataset is successfully loaded, allowing it to be explored and used for training the model.

In [ ]:
```
# You only need to run this if you've imported this notebook to Azure AI Machine Le
# in which case you'll also need to upload the config.json file to the same directo
# and then execute this code to determine the current working directory.
import os
print("Current working directory:", os.getcwd())
print("Files in this directory:", os.listdir())
```

In [ ]:
```
from azureml.core import Datastore

datastore = ws.get_default_datastore()  # workspaceblobstore
datastore.download(target_path='./data', prefix='AI-creditCard')

import pandas as pd
df = pd.read_csv('./data/yourfile.csv')
```

In [ ]:
```
# if you're running locally then use this ...
path = None

# alternatively, if you're running in Azure AI Machine Learning Studio - Notebook,
# (make sure to upload the config.json file to the same directory as this notebook)
#  and then execute this code to determine the current working directory.
#path='Users/[REPLACE-THIS-WITH-YOUR-USERNAME]/config.json'
from azureml.core import Workspace, Dataset
```

```
from azureml.core.authentication import InteractiveLoginAuthentication

# Step 1: Create an auth object
auth = InteractiveLoginAuthentication(force=True)  # optional if multi-tenant

# Step 2: Connect to your workspace using this auth
ws = Workspace.from_config(path="config.json", auth=auth)

# Step 3: Get your dataset
#dataset = Dataset.get_by_name(ws, name='AI-creditCard')
datastore = ws.get_default_datastore()  # workspaceblobstore
datastore.download(target_path='./data', prefix='AI-creditCard')

import pandas as pd
df = pd.read_csv('./data/yourfile.csv')


# Step 4: Load as pandas DataFrame
#df = dataset.to_pandas_dataframe()
print(df.head())


#ws = Workspace.from_config(path=path)
#dataset = Dataset.get_by_name(ws, name='AI-creditCard')
#df = dataset.to_pandas_dataframe()
#df.head()
```

Step 3: Prepare the Data

Intent: Clean and organize the dataset so it is ready for machine learning. This includes standardizing the transaction amount and separating the features from the fraud label.

Expected Outcome: The data is split into:

X – the transaction features used by the model

y – the fraud label indicating whether a transaction is fraudulent

In [ ]:
```
df['Amount'] = (df['Amount'] - df['Amount'].mean()) / df['Amount'].std()
X = df.drop(columns=['Class', 'Time'])
y = df['Class']
```

Step 4: Train the detection model to identify anomalies

Intent: Train the model to identify unusual transactions that may represent fraud.

Expected Outcome: The model learns patterns in the transaction data and produces predictions identifying which transactions appear normal and which appear anomalous.

In [ ]:
```
model = IsolationForest(contamination=0.0017, random_state=42)
model.fit(X)
```

```
y_pred = model.predict(X)
y_pred = [1 if x == -1 else 0 for x in y_pred]
```

Step 5: Evaluate the model's performance

Intent: Measure how well the model identifies fraudulent transactions by comparing its predictions to the known labels.

Expected Outcome: A performance report showing key metrics such as:

Metric	What It Means
Precision	How often the model was correct compared to actual
Recall	How many actual fraud cases was found by the model
F1-Score	Combo of precision and recall that produces a performance score
Support	Number of examples in both the normal and fraud data set

These metrics help determine how effectively the model detects fraud.

Results Summary

Class	Description	Precision	Recall	F1-Score	Support
`0`	Normal transactions	**1.00**	**1.00**	**1.00**	284,315
`1`	Fraudulent transactions	**0.29**	**0.28**	**0.28**	492

In [ ]:
```
# Step 5: Evaluate Model
print(classification_report(y, y_pred))
```

Step 6: Register/publish the trained model (in Azure)

Intent: Save the trained model into Azure Machine Learning so it can be reused.

Expected Outcome: The trained model is stored in the Azure ML workspace and becomes available for future deployment.

In [ ]:
```
import joblib                                          # see https://joblib.readthedoc
                                                       #     Joblib is a set of tools

joblib.dump(model, 'isolation_forest.pkl')
Model.register(model_path='isolation_forest.pkl',
               model_name='creditcard_if_model',
               workspace=ws)
```

Step 7: Create a chart to visualize the results

Results should include a prediction of what "normal" and fraudulent behaviour is.

Intent: Create simple charts that help interpret the model's predictions and understand how anomalies are distributed in the dataset.

Expected Outcome: Chart showing:

- The number of transactions predicted as normal vs anomalous
- How transaction amounts differ between predicted normal and anomalous transactions; if the number of anomalies is close to the number of actual frauds detected in the dataset then the detection model being used is predicting well

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Add predictions to the original dataframe
df['predicted_anomaly'] = y_pred

# Count of predicted anomalies
sns.countplot(x='predicted_anomaly', data=df)
plt.title('Count of Predicted Anomalies')
plt.xlabel('Anomaly (1) vs Normal (0)')
plt.ylabel('Count')
plt.show()
```

Step 7 (continued): Visualize Transaction Amount by Prediction Class

Show a boxplot chart where it shows how the distribution of transaction amounts differs between normal transactions and those flagged as anomalies.

Explanation of the plot:

X-axis: predicted_anomaly

0 = normal transaction

1 = predicted anomaly

Y-axis: Transaction Amount

How to Interpret This Chart

Median comparison:

Median of anomalies is often higher than normal, meaning the model tends to flag large transactions as suspicious.

Spread comparison:

Anomalous transactions often have a wider box, showing more variation in amounts.

Outliers:

Very high-value transactions appear as dots above the whiskers.

These extreme values are often strongly influencing anomaly detection.

In [ ]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='predicted_anomaly', y='Amount')
plt.title('Transaction Amount by Prediction Class')
plt.show()
```

Step 7 (continued): Use SHAP (SHapley Additive exPalanations) to show how the model determined its behaviour, in a visual manner

This chart is generated using SHAP (SHapley Additive exPlanations). It summarizes how each feature (from the dataset) influences model predictions.

Interpeting the SHAP plot

- a dot represents a single transaction.
  - **Red = high** value
  - **Blue = low** value
- each row represents an attribute from the dataset.
- its position on the line show the impact of the model's fraud prediction:
  - Dots farther to the right **push the model toward predicting fraud.**
  - Dots farther to the left **push the model toward predicting normal**.

In [ ]:
```
import shap

explainer = shap.Explainer(model, X)
shap_values = explainer(X[:100])
shap.plots.beeswarm(shap_values)
```

Cost–Benefit Analysis: False Positives vs. Missed Fraud

A. False Positives (Flagging Legitimate Transactions as Fraud)

Costs

- Customer complaints through blocked transactions despite being legitimate; this can lead to a decline in brand reputation and increase customer frustration
- Increased support calls, meaning more overhead
- Lost revenue from blocked transactions

Benefits

- Allows for human review
- Prevents some fraud loss, thus saving revenue
- System is safer to use

B. Missed Fraud

Costs

- Direct financial losses (chargebacks, reimbursements)
- Regulatory penalties (e.g., PCI compliance implications)
- Long-term brand/reputational damage

Benefits

- Smooth customer experience (minimal declined transactions)
- Lower operational overhead

Recommendations for Model Improvement & Deployment (insight from AI)

A. Model Improvements

1. Use Supervised Models (Critical Upgrade)

Isolation Forest is unsupervised and not ideal when labels exist.

Replace or augment with:

Logistic Regression (baseline)

Random Forest

Gradient Boosting (e.g., XGBoost, LightGBM)

👉 These models learn actual fraud patterns, not just anomalies.

2. Handle Class Imbalance

Your dataset:

~0.17% fraud → extremely imbalanced

Apply:

SMOTE (oversampling fraud)

Undersampling normal class

Class weighting

3. Tune Threshold (High Impact)

Isolation Forest uses a fixed contamination threshold.

Instead:

Adjust decision threshold to optimize:

Recall (fraud detection)

Business cost function

4. Feature Engineering

Current features are anonymized → limited signal.

Enhance with:

Transaction velocity (transactions per minute/hour)

Location changes

Merchant category risk

User behavioral patterns

5. Ensemble Approach (Best Practice)

Combine:

Anomaly detection (Isolation Forest)

Supervised classifier

👉 This improves robustness and reduces blind spots.

B. Deployment Recommendations (Azure ML)

1. Real-Time + Batch Hybrid

Real-time scoring → transaction approval/decline

Batch scoring → deeper fraud investigation

2. Human-in-the-Loop System

Send high-risk transactions to analysts

Use feedback to retrain model

3. Model Monitoring (Critical)

Track:

Drift in transaction patterns

Precision/recall over time

Alert volume spikes

Use Azure ML:

Data drift monitors

Model performance dashboards

4. A/B Testing Before Full Rollout

Compare:

Current system vs new model

Measure:

Fraud detection rate

Customer complaints

Revenue impact

Risk Assessment & Mitigation Strategies

Risk Type	Risk Impact	Mitigation
Model	- Poor fraud detection - Overfitting to historical data	- Cross-validation - Regular retraining of model - Benchmark against different models to find best fit
Data	- Changing theme or evolution of fraud - Biased or incomplete data	- Continuous data ingestion - Drift detection - Periodic dataset refresh
Operational	- Not enough manpower to address cases - Growth in transactions increases fraud volume and impacts model performance	- Establish alert priorities - Budget additional compute power - Automate scaling of compute power to compensate for increased demand
Regulatory & Compliance	- Explainability requirements (e.g., financial regulations) - Customer resolution handling	- Use SHAP - Maintain audit logs of decisions
Customer Experience	- False positives introduce frustration with clients	- Greater transparency in transaction and resolution process

Risk Type	Risk Impact	Mitigation
		- More detailed outline of response and decision making

COMMUNICATION PLAN

A. Executives and Senior Leadership

Provide stats/metrics of current detection and its financial impact, with risk exposure. Produce current rate of detection and mitigation strategies

B. Fraud Analysts

Provide stats on alert case loads and accompanying resolutions. Produce suggestions/methods of improvement

C. Data Science / Engineering Teams

Provide feedback to allow for continuous training of detection model.

D. Customers

Provide transparent communications for transactions and disputes, to maintain trust

E. Regulators

Provide logs and decision explainability; maintain constant monitoring and regularly update governance and internal audit methodology